Laboratorijska vježba 1

Provjera sortiranja

Napredni algoritmi i strukture podataka

Opis vježbe

Na vježbi ćete napisati kratak program koji **provjerava** rezultat sortiranja danog niza. Cilj je iskoristiti svojstva sortiranog niza s obzirom na neki referentni niz i provjeriti ta svojstva. Kako je ovo uvodna vježba, točnu specifikaciju svojstava dobit ćete u opisu zadatka kada pristupite samoj vježbi. Imat ćete vremena do kraja termina, a ne dva sata kao na ostalim laboratorijskim vježbama. Za ostale laboratorijske vježbe preporučujemo da varijantu algoritma sami implementirate unaprijed jer će se od vas tražiti neke preinake ili određeni format ulaza i izlaza. Uvijek ćete unaprijed znati koji algoritam i/ili strukturu podataka trebate obraditi, ali nećete dobiti točnu specifikaciju zadatka. Zato ćete na samoj vježbi morati napraviti manje preinake u svojem kodu.

Opis zadatka: Program dobiva ulazni niz i niz za koji netko *tvrdi* da je njegova sortirana inačica te mora odlučiti je li ta tvrdnja točna. Ako tvrdnja nije točna, program mora odrediti koje svojstvo sortiranog niza nije ispunjeno. Točan zadatak te format ulaza i izlaza dobit ćete na samoj vježbi.

Primjer

Neka je ulazni niz $[5, 3, 9, 1]$.

- Tvrdnja $[1, 3, 5, 9]$ je **ispravna**: niz je poredan i sadrži iste elemente kao ulaz.
- Tvrdnja $[3, 1, 5, 9]$ **nije ispravna**: niz sadrži iste elemente, ali nije poredan.
- Tvrdnja $[1, 5, 9]$ također **nije ispravna**.

Literatura

- 1.5.3 Specifications: rules and meaning
- 1.6 Checking an answer you did not produce

Istražite sami

1. **Specifikacija.** Što točno znači da je jedan niz sortirana inačica drugoga? Zapišite to što preciznije.
2. **Protuprimjeri.** Pokušajte „oboriti“ vlastitu definiciju: postoji li izlaz koji je zadovoljava, a ipak nije ispravan?
3. **Eksperiment.** Napišite provjeru i testirajte je na vlastitim primjerima, uključujući neobične i vrlo velike. Na vama je da stvorite i isprobate primjere.

Za razmišljanje (u tom smjeru ići će i pitanja u kvizu).**

- Sortiranje usporedbama zahtijeva $\Omega(n \log n)$ usporedbi. Kolika je cijena *provjere*? Može li provjera biti asimptotski jeftinija od pronalaženja rješenja?
- Radi li vaša provjera ispravno za prazan niz, niz s jednim elementom i niz s elementima koji se ponavljaju?
- Koje operacije u vašoj provjeri izgledaju jeftino, a zapravo prolaze cijeli niz, i koliko vas koštaju kada ih pozivate unutar petlje?
- Koje bi pogrešne izlaze „protivnik“ mogao podmetnuti provjeri koja ne provjerava dovoljno?

** Upravo s ovim pitanjima posebno eksperimentirajte. Na kasnijim vježbama u kvizu možete očekivati i pitanja o tome kako različiti parametri (npr. veličina ulaza ili parametri strukture podataka) utječu na performanse i slično. Važno je da o tim pitanjima stvarno promislite, a ne samo da program prođe testove. Na svakoj laboratorijskoj vježbi ovaj će vam dio biti najvažniji da biste na kvizu dobili što više bodova.

Kako teče vježba

Na vježbi dobivate točan zadatak i svoje pripremljeno rješenje prilagođavate njemu. Program čita sa standardnog ulaza, piše na standardni izlaz i izvodi se na skrivenim testovima. Programski dio možete predati kada vam odgovara, a kratkom kvizu o svojem rješenju i gradivu možete pristupiti **tek kada program prođe sve testove**. Kviz nosi ukupno **2 boda**. Na ovoj vježbi kvizu možete pristupiti *bezbroj* (1000) puta, a uzima se najbolji rezultat.

Dozvoljeni jezici

Jezik	Inačica i opcije prevođenja
Python	3.12.15
C	GCC 15.2.0, `-O2`
C++	GCC 15.2.0, `-O2`, `-std=c++23`
Rust	1.98.1, `--edition=2021`, `opt-level=3`
Java	OpenJDK 21.0.2